



Time Series Anomaly Thresholding and Evaluation in R

Residual Aggregation: L1 and L2 Distance Functions

Residuals from a forecasting model must be aggregated into a single scalar signal before thresholding can identify anomalies. Harbinger provides two utility functions for this via `harutils()`: the L1 (Manhattan) and L2 (Euclidean) distance functions.

```
library(daltoolbox)
library(harbinger)

hutils <- harutils()

# Generate synthetic residuals
values <- rnorm(30, mean = 0, sd = 1)

# L1 residual magnitude aggregation
v1 <- hutils$har_distance_l1(values)
har_plot(harbinger(), v1)

# L2 residual magnitude aggregation
v2 <- hutils$har_distance_l2(values)
har_plot(harbinger(), v2)
```

Step-by-step

harutils()

Instantiates the utility object exposing all helper functions.

rnorm()

Generates synthetic residuals for a controlled demonstration.

har_distance_l1

Sum of absolute deviations; robust to outliers.

har_distance_l2

Sum of squared deviations; sensitive to large residual spikes.

har_plot()

Visualizes the aggregated signal before thresholding.



L1 and L2 are residual aggregations that feed into the threshold stage — choosing between them directly influences which anomalies are surfaced.

L1 Norm (Manhattan Distance)

Calculated as the sum of the absolute values of the individual residual components.

$$\sum_{i=1}^n |r_i|$$

- **Robust to outliers:** Large isolated spikes have less influence on the aggregated signal, since absolute values grow linearly.
- **Treats all deviations equally:** Each residual component contributes proportionally, regardless of magnitude.
- **Best for:** Datasets with occasional large but non-anomalous fluctuations, or when robustness is preferred over sensitivity.

L2 Norm (Euclidean Distance)

Calculated as the square root of the sum of the squared individual residual components.

$$\sqrt{\sum_{i=1}^n r_i^2}$$

- **Sensitive to large deviations:** Squaring residuals amplifies the contribution of large errors, making significant spikes stand out.
- **More "natural" distance:** Corresponds to Euclidean geometry and is widely used in statistical modeling.
- **Best for:** Scenarios where any large deviation is a strong signal of an anomaly or critical event.

Thresholding Strategies for Anomaly Detection

Thresholding converts the continuous scalar residual signal into a binary anomaly decision. Harbinger allows full modularity — strategies are interchangeable via `model$har_outliers`.

Gaussian 3-Sigma Rule

Flags values beyond $\mu \pm 3\sigma$, assuming normal residuals.

- **Pros:** Simple, fast, and widely understood.
- **Cons:** Sensitive to non-normal distributions; can misclassify if data is skewed.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple
hutils <- harutils()

# Baseline: ARIMA + L2 + Gaussian 3-sigma
model <- hanr_arima()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

- ❑ Replacing `model$har_outliers` swaps the threshold rule without touching the rest of the pipeline — full modularity across all event detection workflows.

Boxplot / IQR Method

Flags values outside $Q3 + 1.5 \times IQR$ or $Q1 - 1.5 \times IQR$.

- **Pros:** Robust to skewed distributions and outliers.
- **Cons:** May miss extremes if the distribution is very broad.

```
# Swap in Boxplot/IQR threshold
model <- hanr_arima()
model$har_outliers <- hutils$har_outliers_boxplot
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))

# Swap in Ratio threshold
model <- hanr_arima()
model$har_outliers <- hutils$har_outliers_ratio
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

Ratio-Based Thresholding

Flags values deviating by a relative proportion from a dynamic baseline.

- **Pros:** Effective when residual scale changes over time.
- **Cons:** Requires careful tuning; less precise for stable patterns.

Step-by-step

hanr_arima()

ARIMA-based detector for time series event detection.

har_outliers_boxplot

Threshold via IQR (Tukey, 1977), robust to skewed residuals.

har_outliers_ratio

Threshold via relative deviation from the mean.

attr(detection, "res")

Residuals extracted from the fitted model.

ylines

Renders the threshold boundary on the plot for visual inspection.

Detection Grouping: From Runs to Events

When a threshold is crossed in time series anomaly detection, it often remains crossed for multiple consecutive time steps. This produces a "run" of detections for what is, in essence, a single underlying event, leading to inflated false positive counts and noisy output.

Grouping strategies address this by treating a contiguous block of flagged time steps as a single event. This significantly improves the clarity and actionability of anomaly detection results.

Default: No Grouping

Every flagged time step is reported as a separate detection. This can obscure the true timing of events and lead to an overcount of anomalies, making it difficult to pinpoint the most significant deviation.

Peak selection is critical for downstream evaluation and alerting, as it assigns a precise, single timestamp to each event. This mechanism is controlled via `model$har_outliers_check`, offering modularity akin to other pipeline components.

Grouped: Peak Selection

Using `har_outliers_checks_highgroup`, within each contiguous run, only the time step with the highest residual magnitude (the "peak") is retained as the sole detection point. This provides a precise timestamp for each anomaly.

Combining Distance, Threshold, and Grouping

Building on the previous threshold strategies, these advanced combinations pair L1 distance with different threshold rules — and introduce grouping to retain only the highest-magnitude index within contiguous detection runs.

```
# L1 + Gaussian (default)
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))

# L1 + Boxplot/IQR
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model$har_outliers <- hutils$har_outliers_boxplot
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

```
# L1 + Boxplot + keep only the peak of each group
model <- hanr_arima()
model$har_distance <- hutils$har_distance_l1
model$har_outliers <- hutils$har_outliers_boxplot
model$har_outliers_check <-
  hutils$har_outliers_checks_highgroup
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
har_plot(model, attr(detection,"res"),
  detection, dataset$event,
  yline = attr(detection,"threshold"))
```

Step-by-Step

- **har_distance_l1** — swaps the model's distance function to L1
- **har_outliers_boxplot** — IQR-based threshold combined with L1 distance
- **har_outliers_checks_highgroup** — within contiguous runs, retains only the highest-magnitude change-point
- Each model slot is independently replaceable

📌 **Key concept:** Grouping via **highgroup** prevents multiple event detections for the same anomaly — reducing false positives across contiguous segments of the time series.

Strict Evaluation: Confusion Matrix & har_eval()

Rigorous evaluation is crucial for any event detection system. `har_eval()` computes accuracy, precision, recall, and F1 using a strict criterion — a detection is a true positive only if it aligns **exactly** with a labeled event.

True Positive (TP) A detection that coincides exactly with a labeled event.	False Positive (FP) A detection at a time step where no labeled event exists.	Accuracy $(TP + TN) / (TP + FP + FN + TN)$	Precision $TP / (TP + FP)$ – how many detections are real events.
False Negative (FN) A labeled event that the detector failed to identify.	True Negative (TN) No event and no detection, indicating correct inaction.	Recall $TP / (TP + FN)$ – how many real events were caught.	F1 Score Harmonic mean of precision and recall, balancing both metrics.

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple
head(dataset)
## serie event
## 1.000 FALSE
## 0.969 FALSE

# Configure MLP-based detector
model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 5,
    size = 3,
    decay = 0))
```

```
# Fit model and run detection
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)

# Evaluate with hard metrics
evaluation <- evaluate(har_eval(),
  detection$event,
  dataset$event)

print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 1 0
## FALSE 0 100
```

Step-by-Step

- `head(dataset)` — inspects `serie` and `event` columns
- `hanr_ml(ts_mlp(...))` — MLP-based regression detector
- `ts_norm_gminmax()` — global min-max normalization
- `evaluate(har_eval(), ...)` — computes full confusion matrix
- Result: 1 TP, 0 FP, 0 FN — perfect detection

Key concept:

`har_eval()` enforces strict evaluation — detections must align exactly with labeled events to count as true positives.

Soft Evaluation: Tolerance-Aware Metrics (SoftED)

Strict evaluation penalizes detections that are slightly off as full false positives. SoftED defines a sliding window around each event and assigns partial credit to near-miss detections — better reflecting real-world temporal uncertainty.

Strict Evaluation: Limitations

- Assumes perfect event timing.
- No tolerance for temporal shifts.
- Detection slightly off is a full False Positive.
- Doesn't reflect real-world imprecision.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple

model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 5,
    size = 3,
    decay = 0))
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)

# Hard evaluation (baseline reference)
evaluation <- evaluate(model,
  detection$event, dataset$event,
  evaluation = har_eval())
print(evaluation$confMatrix)
```

SoftED: The Tolerance-Aware Solution

- Defines a sliding window (`sw_size`) around each event.
- Detections within the window receive partial credit.
- Produces "soft" TP, FP, FN for nuanced assessment.
- Better reflects performance with temporal uncertainty.

```
# Soft evaluation with tolerance window sw_size = 5
result <- evaluate(model,
  detection$event, dataset$event,
  evaluation = har_eval_soft(sw_size = 5))
print(result$confMatrix)

# Can also be called without the model object
result <- evaluate(
  har_eval_soft(sw_size = 5),
  detection$event,
  dataset$event)
print(result$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100
```

Step-by-Step

- **har_eval_soft(sw_size = 5)** — applies a tolerance window of ± 5 time points
- Detections falling within the window receive partial credit
- Can be invoked with or without the `model` object
- In this example, hard and soft evaluation yield identical results, since event detection is exact

📌 **Key concept:** SoftED is preferable when temporal uncertainty is present — minor misalignments should not be penalized as outright errors.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

